ECSE 2920: Design Methodology

Deliverable 3

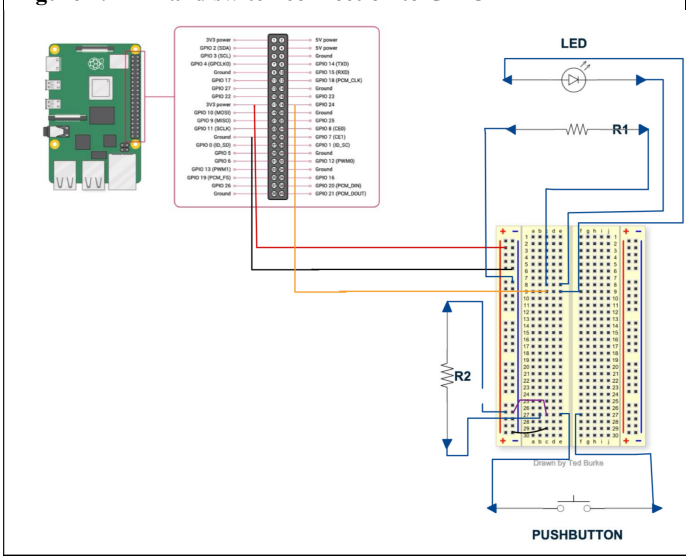
Group 16

01/27/2025

Part 1: Program a switch controlling an LED

Requirements: The requirements for this part consisted of demonstrating GPIO input and output functionality by toggling an LED using a switch, creating a Python script and uploading a 10-second video of the setup in operation, comparing and documenting at least two debounce methods and, explaining the choice for your implementation. The last requirement was drawing and including a circuit diagram in your submission.

Figure 1: LED and switch connection to GPIO



Here is a video demonstrating the LED and switch in action: LED Video

Debounce Method Explanation:

Debouncing is a method used to eliminate unwanted noise or false signals from mechanical switches to ensure reliable input detection. Two methods are software polling with time delay and interruptions with callback functions. Software polling with time delay utilizes the time.sleep() to check the switch state periodically with a small delay. The advantages of this method are that it is simple to implement and requires no advanced hardware or understanding of interrupts. However, it is inefficient as the program halts during the delay and it misses rapid toggles due to periodic checking. Interrupts with callback functions used GPIO.add_event_detect() to trigger a function when the switch state changes. The advantages include that it is more efficient as the program continues to runs other tasks while waiting for an interrupt and it can handle rapid toggles and multiple inputs at the same time. It is also responsive as it reacts immediately to state changes. The disadvantages to this method are that it

is more complex to implement and requires understanding of interrupts and callback mechanisms. Comparing the two methods, software polling is more suitable for simple projects without high demand while interrupts are better for multitasking, fast response, and complex applications.

Key Design Decision(s)

- **What did your team clarify about the design?**
  - We did research for the different debounce methods to see the advantages for each one. During the research, we created Python scripts for the two methods and pushed them to GitHub. We then pulled to the Raspberry Pi and ran the scripts to test circuit functionality.
  - While building the LED circuit, the main issue we had was that our Raspberry Pi extension was not completely pushed to the breadboard, causing our ground pin to be disconnected from the rest of the circuit. To remedy this issue, we talked to the professors and acquired a new Raspberry pi extension to avoid this issue in the future.
- **What were the competing choices in the design?**
  - In terms of the final design, the group used the schematic provided in the RP4 instruction manual as a guide where the RP4 was being tested with the LED and pushbutton through two resistors. However, we had conflicting ideas on LED orientation and where the power source should be connected in the circuit. Ultimately, we did some trial and error (accidentally blew out an LED) and discovered that the negative leg of the LED should be connected to the resistor and the power source should be connected to the positive leg of the LED.
- **Ultimately what did your team choose and why?**
  - In terms of the final design, the group used the schematic provided in the RP4 instruction manual as a guide where the RP4 was being tested with the LED and pushbutton through two resistors. Ultimately, we did some trial and error (accidentally blew out an LED) and discovered that the negative leg of the LED should be connected to the resistor and the power source should be connected to the positive leg of the LED.

Commented [DR1]: need

Commented [DR2]: need

TEST… Test… test

- **What aspects of the design need to be tested?**
  - **Software**
    - The code had to align with the correct GPIO pins for the LED to turn on and off. In terms of the debounce method, the code had to be tested multiple times to check for the sleep function and to observe the delay.
  - **Hardware**

        - We needed to test that the switch was connected correctly, that the LED was connected in the correct orientation, and that the wires were connected to the right channels on the pi extension.
        - We tested both of the code to see the difference in the debounce methods through the button and LED.
- **Who is responsible for testing?**
    - Zane, Disha, Garv
    - **Tests ran?**
        - The test we conducted involved running the Python code in isolation, connecting the LED circuit to an external power source to verify the LED was not burnt out, and running the code with the circuit and adjusting based on errors.
    - **Conclusions from testing?**
        - We needed to connect ground directly from the ground block. We learned from the discussion board that the GND pins do not work [verify lol].
        - The original LED was burned.
        - We needed to get a new pi extension that pushes all the way into the breadboard.

Summary/Part conclusion

For Part 1, the group was assigned to code a LED with a pushbutton for it to function based on when the button is pressed. In terms of the software, multiple code samples were tested to check if the LED and pushbutton have been configured properly and if the code successfully implements the debounce methods provided. In terms of hardware, the LED and the pushbutton had to be properly connected in the breadboard with the resistors for there to a circuit loop that functions. After figuring out that part, the breadboard had to be connected to the pi extension for the code to work in terms of turning the LED on and off when the button is triggered.

Part 2: Start-on-Launch Program

Requirements: The purpose of this section was writing code to configure GPIOs as outputs and make them stay on for a certain amount of time before turning off.

Key Design Decision(s)

- **What did your team clarify about the design?**
  - We originally used the write() function in our code. However, after furhter researched, we discovered it is better to use the output() function for each GPIO, setting to 1, and then 0. Using GPIO.output () helped refer to each pin properly and make sure that each pin is delivered a voltage.
- **What were the competing choices in the design?**
  - We did not have any competing choices in the design, but we discussed what order to light up the GPIO pins – whether to order based on the pi extension we were given or based on the GPIO tester image.
- **Ultimately what did your team choose and why?**
  - We found testing was easier when using the order of the pins on the pi extension, but realized we needed to match the numbers to the GPIO image for the final code.

Commented [MOU3]: Reword

TEST… Test… test

- **What aspects of the design need to be tested?**
  - **Software?**
    - We needed to test that each pin started at 0V, received a voltage of 3.3V, and returned to 0V after the time specified in the sleep() function (we used 5 seconds when testing). Additionally, we also needed to test the code ran on start-up, and did not require us to manually run the script.
- **Who is responsible for testing?**
- Disha and Zane
  - **Tests ran?**
    - **Software:**
      - After running the code, we used a multimeter to touch the positive end to each pin and measured the voltage it received (around 0V, 3.3V, and then 0V). We added print statements to help us understand what part of the script was running and if the appropriate behavior was observed for the pins. We printed “LED ON” after setting the GPIO pin output to high (multimeter should measure 3.3V) and printed “LED OFF” after setting the GPIO pin output to low (multimeter should measure 0V). We also tested to

make sure the program launched when we turned the Raspberry Pi on, and did not require us to manually run the script.

- **Conclusions from testing?**
  - Overall, the tester code worked correctly after replacing the write() function with the GPIO.output() function to light up the GPIO pins.
  - We received an error that saif "No MTA installed" when setting up the launch from startup. After a google search, we found that we needed to add the statement " >dev/null 2>&1" to the crontab –e file to discard at cron tab, which resolved the error.

Commented [MOU4]: Reword

Summary/Part conclusion

We validated the functionality of all GPIOs and ensured the program meets startup and standalone operation requirements. The purpose of this segment was to reboot the pi the correct way where the launch function runs in the correct order. The ideal scenario created for this was that each GPIO pin stays on going one after the other and eventually turns off. From the results, we were able to learn the functionality of the GPIO pins and how the launch of the program will work as we get closer to building the actual design.

Part 3: DC Motors

Requirements: For this section we were required to wiggle two motors clockwise, counterclockwise, and one at a time using our H-bridge in standalone mode.

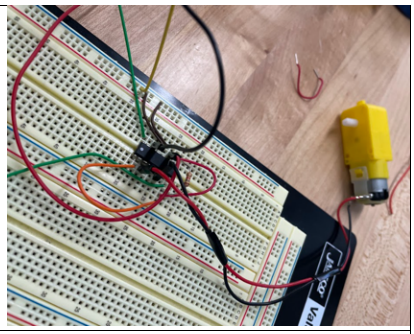


Figure 2: DC Motor connected to H-bridge

Here is a video demonstrating the motors rotating clockwise, counterclockwise, and one at a time using our H-bridge in standalone mode: Motors in action

Key Design Decision(s)

- **What did your team clarify about the design?**
  - We clarified which input terminals need to be high and low to allow the motors to move in either direction or that on the opposite sides of the h-bridge all the terminals are flipped. Additionally, the IC chip and motors each need power provided separately to work properly. The IC chip requires 5V for its internal logic which can be supplied from a DC power supply or the 5V terminal on our PI hat. However, for the standalone mode we used a DC power supply as we were not supposed to use the PI for this task. Furthermore, we discovered that when we supplied more voltage to the motors, they moved faster.
- **What were the competing choices in the design?**
  - We did not have any competing design choices as we simply followed the information provided by the data sheet to complete this task.
- **Ultimately what did your team choose and why?**
  - We all agreed to use the design detailed in our video of motors moving.

TEST… Test… test

- **What aspects of the design need to be tested?**
  - **Software?**
    - Software was not needed for this task as we used the H-bridge in standalone mode.

- **Hardware?**
    - For hardware, we had to test if our H-bridge was correctly driving our motors, if our motors were moving in the correct direction based on the H-bridge input and continuously (not randomly stopping), and we had to test for how changing the signal to the input terminals affected our motors (change in direction and speed).

- **Who is responsible for testing?**

  Garv, Emerson, Zane
    - **Tests ran?**
        - We tested if the H-bridge worked, if the motors worked, and function of input terminals of H-bridge.
    - **Conclusions from testing?**
        - We concluded that our H-bridge was working according to the datasheet (giving the right outputs for our inputs). Our motors were moving correctly and continuously, and when we increased voltage to the motors our speed increased and if we flipped the signals to our input (high and low) direction would change. To change input signals, we would simply take a 5V line coming from a power supply to one of the input terminals and connect it to the other causing a change in direction. If both input terminals on either side of the H-bridge were the same (both high or both low) that motor would not run at all.

Summary/Part Conclusion

The videos for the motors movements are submitted on ELC. We determined that when we increased voltage to the motors, they began to speed up. It took us some experimentation to understand how the input terminals of the H-bridge operate but once we read further into the datasheet, the process was very smooth with no disagreement on how we were to accomplish the task at hand. We found that when a high signal was given to one of the odd input terminals on either side (pins 1A or 3A) the motor would turn counterclockwise and if the even input terminals were high (2A and 4A) the motor would turn clockwise. We observed that both motors moved at different speeds but increasing voltage generally caused them both to speed up. We think that in the future when we use the PI, H-bridge, and motors together we can manipulate the GPIO pins to be high and low to have our car move in the way that we desire.

Part 4: Characterize your Microphone

Requirements: The requirements for this portion were to test and adjust the microphone's gain (sensitivity) for optimal performance and to document results with snapshots of the oscilloscope output.

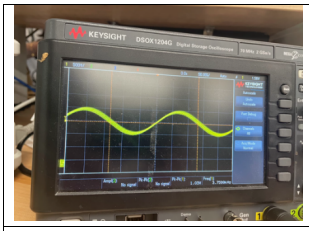


Figure 3: Output for 3686 Hz

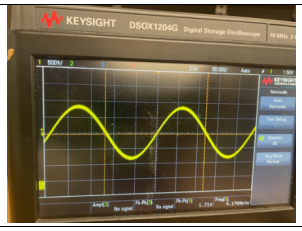


Figure 4: Output for 4186 Hz

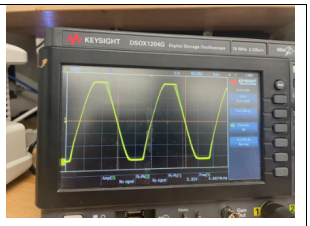


Figure 5: Output for 4686 Hz

Figures 3, 4, and 5 display the output signal from our microphone after inputting sound frequencies 3686 Hz, 4186 Hz, and 4686 Hz respectively. To generate these frequencies, we used a free app called "Tone Gen". We found for all these values; the microphone measured a similar frequency in our output reading. We also tested values going down to 150 Hz and up to 12 kHz, we did not include those images in this report as those frequencies are not relevant for this project. However, we observed that when input frequencies were less than 500 Hz and more than 9 kHz, they would begin to become inaccurate and unstable. This does not concern us because, as stated above, these frequencies are out of scope for this project.

Key Design Decision(s)

- **What did your team clarify about the design?**
  - We needed to solder the microphones as they were not previously soldered, this allowed us to make solid connections for the VCC, GND, and OUT pins so that we could connect them to the breadboard without any faults.
  - We clarified a method for characterizing our microphone, which included using the oscilloscope and a frequency producing app on our phones to measure the required frequency range which was 4186 +/- 500 Hz. The microphones were able to detect the signals with no issue.
- **What were the competing choices in the design?**
  - We were conflicted as a group about whether we should use the spectrum analyzer built into the oscilloscope to gather data on the microphone or to simply observe how the input signal of the microphone changed on the oscilloscope when we provided different input sounds.
- **Ultimately what did your team choose and why?**

Commented [DR5]: explain

Commented [DR6]: what was the method

- After consulting the discussion post on ELC and talking with Dr. Herring, we concluded that our best course of action was to observe how different input sounds affected the output of the microphone on the oscilloscope because our attempts at using the spectrum analyzer for the microphone did not provide us with useful data that we could use to better understand our microphone.

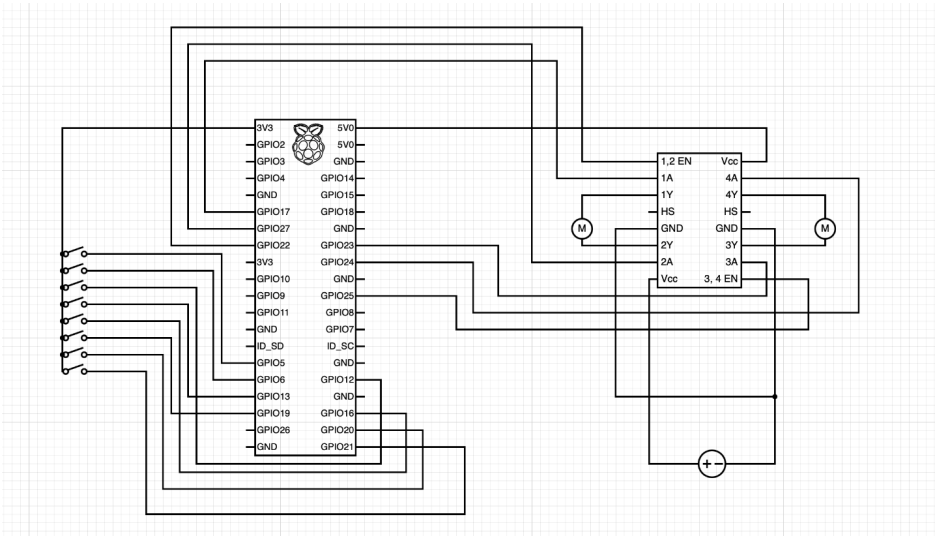
TEST… Test… test

- **<u>What aspects of the design need to be tested?</u>**
  - **Hardware?**
    - We needed to measure the output of the microphone and compare it to what sound frequency we played to gauge how accurate the microphone detected frequencies and how increasing/decreasing gain affects our microphones output wave.
    - We needed to ensure that both microphones functioned similarly, and if they didn't, we needed to make keep that in mind when designing in the future.
- **<u>Who is responsible for testing?</u>**
  Garv, Disha, Zane
  - **Tests ran?**
    - We ran tests to determine how our microphone performed at various frequencies ranging from 150 Hz to 12 kHz.
    - We also ran tests on how increasing gain affected our microphone.
    - We tested to ensure both microphones worked similarly. This means that both microphones should have similar output waves based on the input frequency.
  - **Conclusions from testing?**
    - We determined that when our microphone was given inputs of very high (10 kHz and above) and very low (below 500 Hz) that its output signals began to become inaccurate in detecting the frequencies. When within our C8 range of 4186 Hz (+/- 500 Hz) We found that our signals were very accurate in frequency.
    - When testing our gain increase on the microphone, we discovered that an increase in gain would also increase the output voltage of the microphone and allow it to pick up sounds that were further away from the microphone. When our gain was low, we would have to essentially rest our phones on the microphone to pick up sounds but when gain was increased the source could be slightly further away.
    - When repeating these tests for our second microphone, we found it performed similarly to the first microphone, so both microphones function similarly.

Summary/Part Conclusion:
We ensured our microphones meet our requirements for the project by ensuring that its output frequency and input frequency matched within our C8 range of 4186 Hz (+/- 500 Hz). We also ensured that increasing our gain meant sounds could be picked up from further away. However, we feel that for the project we will likely need to create some sort of funnel to allow sound to be picked up accurately by the microphone from distances that are relatively "far away".

Part 5: Create a Circuit Diagram
Requirements: The requirements for this portion are to create a circuit diagram connecting the dip switch, H-bridge, 2 DC motors, and the raspberry pi 4.

Picture(s)



The circuit is designed to control two DC motors using an 8-switch DIP switch, a Raspberry Pi 4, and an L293D H-Bridge motor driver. Each switch on the DIP switch is connected to a corresponding GPIO pin on the Raspberry Pi, allowing the user to provide binary input signals that determine the operation of the motors. The Raspberry Pi interprets these signals and outputs motor control signals to the L293D motor driver. These control signals are routed to the input pins of the L293D (1A, 2A, 3A, 4A), which drive the motors connected to its output pins (1Y, 2Y, 3Y, 4Y). The enable pins of the L293D are connected to GPIO pins on the Raspberry Pi, allowing the motors to be turned on or off. Power is supplied to the L293D through two Vcc pins: one is connected to the Raspberry Pi's 5V output to power the logic circuitry, while the other is connected to a buck converter stepping down a 24V supply to an appropriate voltage for the motors. The ground pins of the L293D and the buck converter are connected to ensure a common reference for all components. This configuration provides flexibility in controlling

motor direction and speed, with clear wiring between the DIP switch, Raspberry Pi, and motor driver.

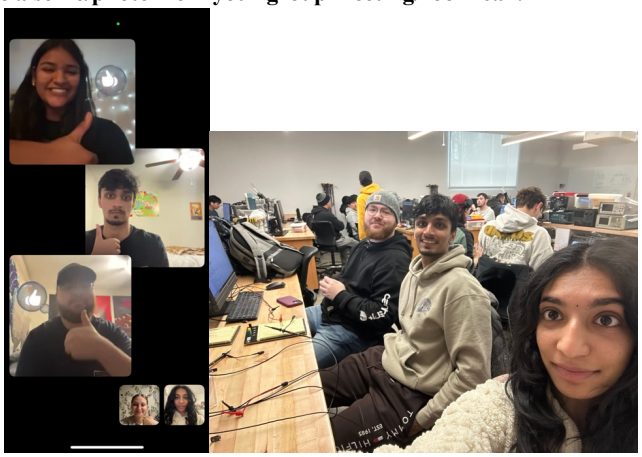
Key Design Decision(s)

- **What did your team clarify about the design?**
    - We clarified that the dip switch would be used to provide user input to enable/disable motors, control their directions, and adjust speed levels.
    - We also clarified improvements to be made from the last H-bridge diagram which includes adding a power supply with a buck converter and connecting the grounds.

Summary/Part Conclusion

The circuit uses a Raspberry Pi 4, an 8-switch DIP switch, and an L293D H-Bridge motor driver to control two DC motors. The DIP switch sends signals to the Raspberry Pi GPIO pins, which generate control signals for the L293D. The L293D drives the motors through its output terminals, with motor enable and direction controlled via Raspberry Pi GPIO pins. The L293D receives logic power from the Raspberry Pi's 5V output and motor power from a buck converter stepping down a 24V supply. Ground connections ensure a shared reference for the entire circuit, enabling efficient and precise motor control.

Conclusion and Participation (REQUIRED)

1. **Include a selfie/photo from your group meeting/zoom call.**
   a. 
2. **When did you meet?**
   a. Thursday, Room 1450, 11:10 am – 12:40 pm & 2:00 pm – 5:45pm
   b. Friday, Room 1409, 8:55 am – 10 am
   c. Tuesday, Room 1409 & 1450, 10:10 am – 12:40 pm, 1:00pm – 1:45pm, 9:00-9:15pm (virtual)
3. **Who was present/not present?**
   a. Thursday – Everyone was present
   b. Friday – Disha, Garv, Zane were present.
   c. Tuesday – Everyone was present.
4. **What were the main ideas discussed or major decisions (1-2 sentences/bullet points)**
   a. For LED functionality, we need to replace the extension with one that connects to breadboard better.
   b. The conclusions from the spectrum analyzer analysis were that we need to work on detecting from distances and need to solder the microphones.